KAFKA EM AMBIENTE BIG TECH

Guia completo — KRaft · SSL · SASL · Producer · Consumer · Prometheus · Grafana
Java Spring · Docker Compose · Simulação de produção real

Arquitetura do sistema

Em uma big tech real o Kafka roda em pelo menos 3 servidores físicos separados — um broker por servidor. O producer e o consumer são microserviços independentes, cada um em seu próprio container. O Prometheus coleta métricas de todos os serviços e o Grafana exibe os dashboards.

#	Serviço	Onde roda / Porta
1	Kafka Broker 1	`Servidor 1 — IP1:9092 (broker) e IP1:9093 (controller)`
2	Kafka Broker 2	`Servidor 2 — IP2:9092 (broker) e IP2:9093 (controller)`
3	Kafka Broker 3	`Servidor 3 — IP3:9092 (broker) e IP3:9093 (controller)`
4	Producer	`Servidor App — porta 8080`
5	Consumer	`Servidor App — porta 8081`
6	Prometheus	`Servidor Monitor — porta 9090`
7	Grafana	`Servidor Monitor — porta 3000`

> *Para simular localmente: substitua IP1, IP2, IP3 pelo IP da sua VM e use KAFKA_NODE_ID=1, 2 e 3 em containers separados na mesma rede Docker.*

Parte 1 — Rede Docker e certificados SSL

Antes de subir qualquer broker, crie a rede e gere os certificados SSL. Todo o tráfego Kafka em produção é criptografado — sem SSL qualquer pessoa na rede consegue ler as mensagens com Wireshark.

1.1 — Criar a rede Docker

```
docker network create kafka-net
```

1.2 — Criar as pastas de dados com permissão correta

```
# Execute em cada servidor antes de subir o broker
mkdir -p /data/kafka1   # no Servidor 1
mkdir -p /data/kafka2   # no Servidor 2
mkdir -p /data/kafka3   # no Servidor 3
chmod 777 /data/kafka1
chmod 777 /data/kafka2
chmod 777 /data/kafka3
```

```
# Pasta para os certificados SSL
mkdir -p /etc/kafka/ssl
chmod 755 /etc/kafka/ssl
```

1.3 — Gerar os certificados SSL

Execute estes comandos uma vez, no Servidor 1. Os certificados gerados são copiados para os outros servidores depois.

Passo A — Criar a autoridade certificadora (CA)

```
# Gera a chave privada da CA
openssl genrsa -out /etc/kafka/ssl/ca.key 4096

# Gera o certificado da CA (válido por 10 anos)
openssl req -new -x509 -days 3650 \
  -key /etc/kafka/ssl/ca.key \
  -out /etc/kafka/ssl/ca.crt \
  -subj "/CN=kafka-ca/OU=platform/O=suaempresa/C=BR"
```

Passo B — Criar certificado para cada broker

```
# Repita este bloco para broker1, broker2 e broker3
# Substituindo 'broker1' e 'IP1' pelo número e IP correto

# Gera chave privada do broker
openssl genrsa -out /etc/kafka/ssl/broker1.key 2048

# Gera requisição de assinatura (CSR)
openssl req -new \
  -key /etc/kafka/ssl/broker1.key \
  -out /etc/kafka/ssl/broker1.csr \
  -subj "/CN=broker1/OU=kafka/O=suaempresa/C=BR"

# Assina o certificado com a CA
openssl x509 -req -days 365 \
  -in  /etc/kafka/ssl/broker1.csr \
  -CA  /etc/kafka/ssl/ca.crt \
  -CAkey /etc/kafka/ssl/ca.key \
  -CAcreateserial \
  -out /etc/kafka/ssl/broker1.crt

# Cria o keystore PKCS12 do broker (senha: kafka-ssl-senha)
openssl pkcs12 -export \
  -in  /etc/kafka/ssl/broker1.crt \
  -inkey /etc/kafka/ssl/broker1.key \
  -name broker1 \
  -out /etc/kafka/ssl/broker1.p12 \
  -passout pass:kafka-ssl-senha

# Importa para o formato JKS que o Kafka usa
keytool -importkeystore \
  -srckeystore  /etc/kafka/ssl/broker1.p12 \
  -srcstoretype PKCS12 \
  -srcstorepass kafka-ssl-senha \
  -destkeystore /etc/kafka/ssl/broker1.keystore.jks \
  -deststoretype JKS \
  -deststorepass kafka-ssl-senha \
  -noprompt

# Importa a CA no truststore compartilhado
keytool -importcert \
  -alias ca-root \
  -file /etc/kafka/ssl/ca.crt \
```

```
  -keystore /etc/kafka/ssl/kafka.truststore.jks \
  -storepass kafka-ssl-senha \
  -noprompt
```

> *Repita o Passo B trocando broker1/IP1 por broker2/IP2 e broker3/IP3. O truststore (kafka.truststore.jks) é o mesmo para todos.*

Passo C — Copiar certificados para os outros servidores

```
# Do Servidor 1 para o Servidor 2
scp /etc/kafka/ssl/broker2.keystore.jks  usuario@IP2:/etc/kafka/ssl/
scp /etc/kafka/ssl/kafka.truststore.jks   usuario@IP2:/etc/kafka/ssl/

# Do Servidor 1 para o Servidor 3
scp /etc/kafka/ssl/broker3.keystore.jks  usuario@IP3:/etc/kafka/ssl/
scp /etc/kafka/ssl/kafka.truststore.jks   usuario@IP3:/etc/kafka/ssl/
```

Parte 2 — Kafka KRaft com SSL e SASL

2.1 — Gerar o Cluster ID (uma única vez)

```
# Execute só uma vez — no Servidor 1
docker run --rm confluentinc/cp-kafka:7.5.0 kafka-storage random-uuid
# Exemplo de saída: q1Sh-9_ISia_zwGINzRvyQ
# Guarde este valor — será usado em TODOS os brokers
```

2.2 — Formatar o volume de cada broker

```
# Servidor 1 — substitua SEU_UUID pelo valor gerado acima
docker run --rm \
  -v /data/kafka1:/var/lib/kafka/data \
  -v /etc/kafka/ssl:/etc/kafka/ssl \
  -e CLUSTER_ID=SEU_UUID \
  confluentinc/cp-kafka:7.5.0 \
  bash -c "kafka-storage format \
    --config /etc/kafka/kraft/server.properties \
    --cluster-id SEU_UUID"

# Repita nos Servidores 2 e 3 trocando /data/kafka1 por /data/kafka2 e kafka3
```

2.3 — Subir os 3 brokers

Execute cada comando no servidor correspondente. Todos usam o mesmo CLUSTER_ID.

Broker 1 — Servidor 1

```
docker run -d \
  --name kafka1 \
  --network kafka-net \
  --restart unless-stopped \
  -p 9092:9092 -p 9093:9093 \
  -v /data/kafka1:/var/lib/kafka/data \
  -v /etc/kafka/ssl:/etc/kafka/ssl \
  -e CLUSTER_ID=SEU_UUID \
  -e KAFKA_NODE_ID=1 \
```

```
  -e KAFKA_PROCESS_ROLES=broker,controller \
  -e KAFKA_LISTENERS=SSL://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093 \
  -e KAFKA_ADVERTISED_LISTENERS=SSL://IP1:9092 \
  -e KAFKA_LISTENER_SECURITY_PROTOCOL_MAP=SSL:SSL,CONTROLLER:PLAINTEXT \
  -e KAFKA_CONTROLLER_LISTENER_NAMES=CONTROLLER \
  -e KAFKA_CONTROLLER_QUORUM_VOTERS=1@IP1:9093,2@IP2:9093,3@IP3:9093 \
  -e KAFKA_LOG_DIRS=/var/lib/kafka/data \
  -e KAFKA_SSL_KEYSTORE_FILENAME=broker1.keystore.jks \
  -e KAFKA_SSL_KEYSTORE_CREDENTIALS=ssl_senha \
  -e KAFKA_SSL_KEY_CREDENTIALS=ssl_senha \
  -e KAFKA_SSL_TRUSTSTORE_FILENAME=kafka.truststore.jks \
  -e KAFKA_SSL_TRUSTSTORE_CREDENTIALS=ssl_senha \
  -e KAFKA_SSL_CLIENT_AUTH=required \
  -e KAFKA_SECURITY_INTER_BROKER_PROTOCOL=SSL \
  -e KAFKA_SASL_ENABLED_MECHANISMS=PLAIN \
  -e KAFKA_SASL_MECHANISM_INTER_BROKER_PROTOCOL=PLAIN \
  -e KAFKA_OPTS="-Djava.security.auth.login.config=/etc/kafka/ssl/kafka_server_jaas.conf" \
  -e KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=3 \
  -e KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR=3 \
  -e KAFKA_TRANSACTION_STATE_LOG_MIN_ISR=2 \
  -e KAFKA_MIN_INSYNC_REPLICAS=2 \
  confluentinc/cp-kafka:7.5.0
```

> *Repita para Broker 2 e 3 trocando: --name kafka2/kafka3, KAFKA_NODE_ID=2/3, KAFKA_SSL_KEYSTORE_FILENAME=broker2/3.keystore.jks, /data/kafka2 e /data/kafka3.*

2.4 — Arquivo JAAS para autenticação SASL

Crie este arquivo em /etc/kafka/ssl/kafka_server_jaas.conf em todos os servidores Kafka:

```
KafkaServer {
  org.apache.kafka.common.security.plain.PlainLoginModule required
  username="admin"
  password="senha-admin-forte"
  user_admin="senha-admin-forte"
  user_producer="senha-producer-forte"
  user_consumer="senha-consumer-forte";
};
```

> *Use senhas fortes em produção — nunca use 'senha123' ou valores óbvios. Guarde as senhas em um cofre de segredos como HashiCorp Vault ou AWS Secrets Manager.*

2.5 — Criar os tópicos

```
docker exec -it kafka1 bash

# Tópico principal — imposto de renda
kafka-topics \
  --create \
  --topic verificacao-imposto \
  --partitions 10 \
  --replication-factor 3 \
  --config retention.ms=604800000 \
  --bootstrap-server IP1:9092

# Tópico de erros (DLT)
kafka-topics \
  --create \
  --topic verificacao-imposto.DLT \
  --partitions 10 \
  --replication-factor 3 \
  --config retention.ms=604800000 \
  --bootstrap-server IP1:9092
```

```
# Confirma
kafka-topics --list --bootstrap-server IP1:9092

exit
```

Parte 3 — Producer Java Spring

O producer é um microserviço Spring Boot que recebe dados do sistema bancário via HTTP POST e os publica no Kafka. Você altera apenas as variáveis marcadas.

3.1 — Variáveis que você muda por projeto

Variável	O que é / Exemplo
kafka.broker	`IP1:9092,IP2:9092,IP3:9092`
kafka.topico	`verificacao-imposto`
kafka.chave-campo	`conta_id (campo que define a partição)`
kafka.ssl.truststore	`/etc/kafka/ssl/kafka.truststore.jks`
kafka.ssl.truststore.senha	`kafka-ssl-senha`
kafka.sasl.usuario	`producer`
kafka.sasl.senha	`senha-producer-forte`
server.port	`8080`

3.2 — application.properties do producer

```
# ============================================================
# ALTERE APENAS ESTA SEÇÃO
# ============================================================
kafka.broker=IP1:9092,IP2:9092,IP3:9092
kafka.topico=verificacao-imposto
kafka.chave-campo=conta_id
server.port=8080
spring.application.name=producer-imposto

# SSL — criptografa o tráfego
kafka.ssl.truststore=/etc/kafka/ssl/kafka.truststore.jks
kafka.ssl.truststore.senha=kafka-ssl-senha

# SASL — autenticação no broker
kafka.sasl.usuario=producer
kafka.sasl.senha=senha-producer-forte

# Actuator — expõe métricas para o Prometheus
management.endpoints.web.exposure.include=health,metrics,prometheus
management.endpoint.health.show-details=always

# Logs
logging.level.com.producer=INFO
logging.level.org.apache.kafka=WARN
```

3.3 — KafkaProducerConfig.java — bloco SSL+SASL

Adicione estas configurações dentro do método producerFactory(), depois das configs de performance:

```
// SSL — criptografia do tráfego
config.put("security.protocol", "SASL_SSL");
config.put("ssl.truststore.location", truststoreLocation);
config.put("ssl.truststore.password", truststoreSenha);
config.put("ssl.endpoint.identification.algorithm", "https");

// SASL — autenticação no broker
config.put("sasl.mechanism", "PLAIN");
config.put("sasl.jaas.config",
    "org.apache.kafka.common.security.plain.PlainLoginModule required" +
    " username=\"" + saslUsuario + "\"" +
    " password=\"" + saslSenha + "\";");
```

3.4 — Campos da classe Evento por projeto

Mude os campos da classe Evento para bater com os dados do seu sistema bancário:

```
class Evento {
    private String transacao_id;   // gerado automaticamente
    private String timestamp;      // gerado automaticamente

    // === CAMPOS DO SEU PROJETO — altere aqui ===
    private String conta_id;       // identifica o usuário (chave da partição)
    private String cpf;            // CPF do contribuinte
    private String ano_referencia; // ex: "2025"
    private String tipo_consulta;  // ex: "IMPOSTO_RENDA"
    // ============================================

    // getters e setters manuais para cada campo
    public String getConta_id() { return conta_id; }
    public void setConta_id(String v) { this.conta_id = v; }
    // ...
}
```

3.5 — Dockerfile do producer

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -q
COPY src ./src
RUN mvn clean package -DskipTests -q

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/producer-imposto-1.0.0.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Parte 4 — Consumer Java Spring

4.1 — Variáveis que você muda por projeto

Variável	O que é / Exemplo

kafka.broker	IP1:9092,IP2:9092,IP3:9092
kafka.topico	verificacao-imposto
kafka.grupo	grupo-verificacao-imposto
kafka.consumer.concurrency	10 (igual ao número de partições)
kafka.dlq.tentativas	3 (tentativas antes de ir para o DLT)
kafka.ssl.truststore	/etc/kafka/ssl/kafka.truststore.jks
kafka.ssl.truststore.senha	kafka-ssl-senha
kafka.sasl.usuario	consumer
kafka.sasl.senha	senha-consumer-forte
server.port	8081

4.2 — application.properties do consumer

```
# ============================================================
# ALTERE APENAS ESTA SEÇÃO
# ============================================================
kafka.broker=IP1:9092,IP2:9092,IP3:9092
kafka.topico=verificacao-imposto
kafka.grupo=grupo-verificacao-imposto
kafka.destino.url=http://api-resultado:5000/resultado
kafka.dlq.tentativas=3
kafka.consumer.concurrency=10
server.port=8081
spring.application.name=consumer-imposto

# SSL
kafka.ssl.truststore=/etc/kafka/ssl/kafka.truststore.jks
kafka.ssl.truststore.senha=kafka-ssl-senha

# SASL
kafka.sasl.usuario=consumer
kafka.sasl.senha=senha-consumer-forte

# Actuator
management.endpoints.web.exposure.include=health,metrics,prometheus

# Logs
logging.level.com.example=INFO
logging.level.org.apache.kafka=WARN
```

4.3 — KafkaConsumerConfig.java — bloco SSL+SASL

Adicione dentro do método consumerFactory(), depois das configs padrão:

```
// SSL
props.put("security.protocol", "SASL_SSL");
props.put("ssl.truststore.location", truststoreLocation);
props.put("ssl.truststore.password", truststoreSenha);
props.put("ssl.endpoint.identification.algorithm", "https");

// SASL
props.put("sasl.mechanism", "PLAIN");
props.put("sasl.jaas.config",
    "org.apache.kafka.common.security.plain.PlainLoginModule required" +
    " username=\"" + saslUsuario + "\"" +
    " password=\"" + saslSenha + "\";");
```

4.4 — Lógica de negócio em processarEvento()

Este é o único método onde você coloca a lógica específica do projeto. O resto do consumer não muda:

```
public ResultadoEvento processarEvento(Evento evento) {

    // === SUA LÓGICA DE NEGÓCIO AQUI ===
    // Exemplo: verificar imposto do usuário

    String contaId = evento.getConta_id();
    String ano     = evento.getAno_referencia();

    // Consulta seu sistema interno (banco de dados, API legada, etc.)
    boolean pagouImposto = sistemaFiscal.verificar(contaId, ano);

    return ResultadoEvento.builder()
        .id(contaId)
        .status(pagouImposto ? "REGULAR" : "PENDENTE")
        .mensagem(pagouImposto
            ? "Imposto quitado para " + ano
            : "Imposto pendente para " + ano)
        .processado_em(Instant.now().toString())
        .build();
    // ===================================
}
```

Parte 5 — Docker Compose completo

Um arquivo docker-compose.yml por grupo de serviços. Em produção real cada grupo fica em servidores diferentes.

5.1 — docker-compose.yml dos brokers Kafka

Coloque em cada servidor Kafka. Troque os IPs e nomes conforme o servidor.

```
version: "3.9"
services:
  kafka1:                          # troque por kafka2/kafka3 nos outros servidores
    image: confluentinc/cp-kafka:7.5.0
    container_name: kafka1
    restart: unless-stopped
    network_mode: host             # usa a rede do servidor diretamente
    volumes:
      - /data/kafka1:/var/lib/kafka/data
      - /etc/kafka/ssl:/etc/kafka/ssl
    environment:
      CLUSTER_ID: SEU_UUID
      KAFKA_NODE_ID: 1             # 2 e 3 nos outros servidores
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: SSL://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093
      KAFKA_ADVERTISED_LISTENERS: SSL://IP1:9092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: SSL:SSL,CONTROLLER:PLAINTEXT
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@IP1:9093,2@IP2:9093,3@IP3:9093
      KAFKA_LOG_DIRS: /var/lib/kafka/data
      KAFKA_SSL_KEYSTORE_FILENAME: broker1.keystore.jks
      KAFKA_SSL_KEYSTORE_CREDENTIALS: ssl_senha
      KAFKA_SSL_KEY_CREDENTIALS: ssl_senha
      KAFKA_SSL_TRUSTSTORE_FILENAME: kafka.truststore.jks
      KAFKA_SSL_TRUSTSTORE_CREDENTIALS: ssl_senha
      KAFKA_SSL_CLIENT_AUTH: required
      KAFKA_SECURITY_INTER_BROKER_PROTOCOL: SSL
```

```
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 3
      KAFKA_MIN_INSYNC_REPLICAS: 2
```

5.2 — docker-compose.yml do servidor de aplicação

```
version: "3.9"
services:
  producer:
    build:
      context: ./producer
    container_name: producer-imposto
    restart: unless-stopped
    ports:
      - "8080:8080"
    volumes:
      - /etc/kafka/ssl:/etc/kafka/ssl
    env_file: .env                  # senhas ficam no .env, nunca no compose

  consumer:
    build:
      context: ./consumer
    container_name: consumer-imposto
    restart: unless-stopped
    ports:
      - "8081:8081"
    volumes:
      - /etc/kafka/ssl:/etc/kafka/ssl
    env_file: .env
```

5.3 — docker-compose.yml do Prometheus e Grafana

```
version: "3.9"
services:
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    restart: unless-stopped
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
      - prometheus-data:/prometheus
    command:
      - --config.file=/etc/prometheus/prometheus.yml
      - --storage.tsdb.retention.time=30d

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    restart: unless-stopped
    ports:
      - "3000:3000"
    volumes:
      - grafana-data:/var/lib/grafana
    environment:
      GF_SECURITY_ADMIN_PASSWORD: senha-grafana-forte

volumes:
  prometheus-data:
  grafana-data:
```

Parte 6 — Prometheus e Grafana

6.1 — prometheus.yml

Crie este arquivo na mesma pasta do docker-compose do monitoramento:

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:

  # Coleta métricas do producer
  - job_name: producer
    static_configs:
      - targets: ["IP_SERVIDOR_APP:8080"]
    metrics_path: /actuator/prometheus

  # Coleta métricas do consumer
  - job_name: consumer
    static_configs:
      - targets: ["IP_SERVIDOR_APP:8081"]
    metrics_path: /actuator/prometheus

  # Coleta métricas do próprio Prometheus
  - job_name: prometheus
    static_configs:
      - targets: ["localhost:9090"]
```

6.2 — Dependências obrigatórias no pom.xml do producer e consumer

Adicione no pom.xml de ambos para que o Prometheus consiga coletar métricas:

```
<!-- Actuator — expõe o endpoint /actuator/prometheus -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<!-- Micrometer — formata as métricas no padrão Prometheus -->
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

6.3 — Dashboards no Grafana

Após subir o Grafana em http://IP_MONITOR:3000 (admin / senha-grafana-forte):

#	O que fazer	Caminho no Grafana
1	Adicionar Prometheus como fonte de dados	`Configuration → Data Sources → Add → Prometheus → URL: http://prometheus:9090`
2	Importar dashboard do Kafka	`Dashboards → Import → ID: 7589 (Kafka Overview)`
3	Importar dashboard do Spring Boot	`Dashboards → Import → ID: 12900 (Spring Boot)`

4	Criar alerta de lag do consumer	`Alerting → New Alert Rule → kafka_consumer_lag > 1000`

6.4 — Métricas mais importantes para monitorar

Métrica	O que significa
kafka_producer_record_send_rate	`Mensagens publicadas por segundo pelo producer`
kafka_consumer_fetch_manager_records_consumed_rate	`Mensagens consumidas por segundo`
kafka_consumer_coordinator_commit_latency_avg	`Latência média do commit de offset`
kafka_producer_record_error_rate	`Taxa de erros de publicação — deve ser zero`
jvm_memory_used_bytes	`Memória RAM usada pelo producer/consumer`
http_server_requests_seconds_count	`Número de requisições HTTP recebidas`

Parte 7 — Ordem de execução completa

> *Sempre siga esta ordem. O Kafka deve estar no ar antes do consumer. O consumer deve estar no ar antes do producer receber tráfego.*

#	Onde executar	Comando / Ação
1	Todos os servidores	`mkdir -p /data/kafkaX && chmod 777 /data/kafkaX`
2	Servidor 1	`Gerar certificados SSL e copiar para Servidores 2 e 3`
3	Servidor 1	`docker run random-uuid → guardar o UUID`
4	Todos os servidores Kafka	`docker run formatar volume com o UUID`
5	Todos os servidores Kafka	`docker-compose up -d (sobe os brokers)`
6	Servidor 1	`docker exec kafka1 → criar tópicos → exit`
7	Servidor Monitor	`docker-compose up -d (sobe Prometheus + Grafana)`
8	Servidor App	`docker-compose up -d --build (sobe producer + consumer)`
9	Navegador	`http://IP_MONITOR:3000 → verificar dashboards`
10	Teste	`curl POST ao producer → verificar log do consumer → GET resultado`

Resumo — o que você muda por projeto

Em todos os projetos Kafka a infraestrutura é sempre a mesma. O que muda são apenas estes itens:

O que muda	Onde mudar
Nome do tópico	`kafka.topico em ambos application.properties + comando kafka-topics`
Campos da classe Evento	`Classe Evento no EventoProducer.java — getters e setters manuais`
Lógica de negócio	`Método processarEvento() no EventoConsumer.java`
Concurrency do consumer	`kafka.consumer.concurrency = número de partições do tópico`
IPs dos brokers	`kafka.broker em ambos application.properties`
Credenciais SASL	`kafka.sasl.usuario e kafka.sasl.senha em ambos + kafka_server_jaas.conf`
Senha SSL	`kafka.ssl.truststore.senha em ambos + geração dos certificados`
Nome do grupo	`kafka.grupo no consumer — um nome diferente por serviço`

Para usar prompt de producer e consumer para IA.

==========

PRODUCER PROMPT:

==========

You are a senior Kafka engineer. I have a working Kafka Producer service
built with Java Spring Boot 3 and Spring Kafka. I need you to adapt it
for a new project.

Current file structure of this project (do not change the structure)

```
producer/
├── src/main/java/com/producer/EventoProducer.java   ← single file, all classes
├── src/main/resources/application.properties
├── pom.xml
└── Dockerfile
```

What EventoProducer.java contains (do not change this structure)

All classes are in a single file in this exact order:

1. EventoProducerApplication → @SpringBootApplication entry point
2. Evento → model with manual getters/setters (no Lombok)
3. EventoController → @RestController with POST /api/eventos and GET /api/health
4. EventoProducerService → @Service with publicarEvento(), validarEvento(), preencherCamposAutomaticos(), enviarComCallback()
5. KafkaProducerConfig → @Configuration with producerFactory() and kafkaTemplate()

6. CorsConfig → @Configuration implements WebMvcConfigurer

Infrastructure

Kafka brokers: [IP:9092] ← change this
  For multiple brokers: IP1:9092,IP2:9092,IP3:9092
Kafka version: confluentinc/cp-kafka:7.5.0
SSL: [yes/no] ← change this
SASL: [yes/no] ← change this

If SSL=yes, add these configs in producerFactory():

```
  config.put("security.protocol", "SASL_SSL");
  config.put("ssl.truststore.location", truststoreLocation);
  config.put("ssl.truststore.password", truststoreSenha);
  config.put("ssl.endpoint.identification.algorithm", "https");
  config.put("sasl.mechanism", "PLAIN");
  config.put("sasl.jaas.config", "org.apache.kafka.common.security.plain
          .PlainLoginModule required username=\"USER\" password=\"PASS\";");
```

Project settings

Topic name: [TOPIC NAME] ← change this
Server port: [PORT] ← change this (default: 8080)
Frontend origin for CORS: [URL] ← change this (example: http://localhost:3000)
Partition key field: [FIELD NAME] ← change this
  This field value becomes the Kafka message key.
  Events with the same key always go to the same partition (order guaranteed).
  Example: conta_id, user_id, order_id

Evento model — fields that arrive from the frontend via POST /api/eventos

These two fields are always present and auto-generated — keep them:
  - String transacao_id (auto-generated UUID if not provided)
  - String timestamp (auto-generated ISO-8601 if not provided)

Add these business fields (replace the examples with your actual fields):
[LIST YOUR FIELDS HERE] ← change this
Example:
  - String conta_id (partition key)
  - String cpf
  - BigDecimal valor
  - String tipo (expected: PIX, TED, DOC, BOLETO)
  - String descricao

For each field generate:
  - private declaration
  - public getter
  - public setter
  - include in toString()

Validation rules — validarEvento() in EventoProducerService

Replace the validation body with these rules:
[DESCRIBE YOUR VALIDATION RULES] ← change this
Example:
- conta_id is required (throw IllegalArgumentException if blank)
- valor must be > 0.01 (throw IllegalArgumentException if not)
- tipo must be one of: PIX, TED, DOC, BOLETO (throw IllegalArgumentException if not)

Rules that throw IllegalArgumentException return HTTP 400 to the caller.
Rules that throw any other exception return HTTP 500.

application.properties settings

Generate the properties file with a section clearly marked
"ALTERE APENAS AQUI" containing:
kafka.broker = [the brokers you specified above]
kafka.topico = [the topic name you specified above]
kafka.chave-campo = [the partition key field you specified above]
server.port = [the port you specified above]
Plus SSL/SASL properties if SSL=yes

Important constraints — never violate these

1. Package declaration must be the first line:
   package com.producer;

2. Do NOT use any Lombok annotation:
   No @Data, @Slf4j, @Builder, @RequiredArgsConstructor, @AllArgsConstructor
   Use instead:
   - Manual getters and setters
   - private static final Logger log = LoggerFactory.getLogger(ClassName.class);
   - Manual constructors where needed

3. Do NOT include spring-boot-starter-security in pom.xml.
   It blocks all HTTP requests with 401 by default.

4. The main class must NOT be public:
   class EventoProducerApplication (not public class)

5. Keep all classes in the single file EventoProducer.java.
   Do not split into multiple files.

6. CorsConfig must always be included — without it the browser
   blocks the POST request with CORS error.

7. The pom.xml artifactId must be: kafka-evento-producer
   The pom.xml version must be: 1.0.0
   The Dockerfile must copy: target/kafka-evento-producer-1.0.0.jar

8. The Dockerfile must use two-stage build:
   Stage 1: maven:3.9-eclipse-temurin-17 to compile
   Stage 2: eclipse-temurin:17-jre-alpine to run
   This way the JAR is compiled inside Docker — no need to run mvn locally.

Output format

Generate these 3 files completely, in this order:

FILE 1: src/main/java/com/producer/EventoProducer.java
  Complete file, all 6 classes, ready to copy and paste.
  Comments in Portuguese.

FILE 2: src/main/resources/application.properties
  Complete file with "ALTERE APENAS AQUI" section at the top.
  Comments in Portuguese explaining each property.

FILE 3: pom.xml
  Complete file with all required dependencies.
  No spring-boot-starter-security.
  Comments in Portuguese.

Do not generate the Dockerfile — it never changes between projects.

==========

CONSUMER PROMPT:

==========

You are a senior Kafka engineer. I have a working Kafka Consumer service
built with Java Spring Boot 3 and Spring Kafka. I need you to adapt it
for a new project.

Current file structure of this project (do not change the structure)

```
consumer/
├── src/main/java/com/example/consumer/EventoConsumer.java  ← single file, all classes
├── src/main/resources/application.properties
├── pom.xml
└── Dockerfile
```

What EventoConsumer.java contains (do not change this structure)

All classes are in a single file in this exact order:
  1. EventoConsumerApp → @SpringBootApplication entry point
  2. Evento → model matching the producer (manual getters/setters)
  3. ResultadoEvento → result model returned to frontend (manual getters/setters)
  4. ResultadoStore → @Service with ConcurrentHashMap (in-memory store)
  5. ResultadoController → @RestController with GET /api/resultado/{id},
     DELETE /api/resultado/{id}, GET /api/health
  6. EventoConsumerService → @Service with processarEvento() and enviarParaServico()
  7. EventoConsumer → @Service with @KafkaListener and manual offset commit

8. KafkaConsumerConfig → @Configuration with consumerFactory(), errorHandler(),
kafkaListenerContainerFactory(), producerFactory(),
kafkaTemplate() for DLT
9. CorsConfig → @Configuration implements WebMvcConfigurer

Infrastructure

Kafka brokers: [IP:9092] ← change this
For multiple brokers: IP1:9092,IP2:9092,IP3:9092
Kafka version: confluentinc/cp-kafka:7.5.0
SSL: [yes/no] ← change this
SASL: [yes/no] ← change this

If SSL=yes, add these configs in both consumerFactory() and producerFactory():

```
props.put("security.protocol", "SASL_SSL");
props.put("ssl.truststore.location", truststoreLocation);
props.put("ssl.truststore.password", truststoreSenha);
props.put("ssl.endpoint.identification.algorithm", "https");
props.put("sasl.mechanism", "PLAIN");
props.put("sasl.jaas.config", "org.apache.kafka.common.security.plain
        .PlainLoginModule required username=\"USER\" password=\"PASS\";");
```

Project settings

Topic name: [TOPIC NAME] ← change this
Consumer group: [GROUP NAME] ← change this
Use a unique name per service. Example: grupo-processamento-pix
Concurrency: [NUMBER] ← change this
Must equal the number of partitions of the topic.
Example: topic with 9 partitions → concurrency=9
DLT retries: [NUMBER] ← change this (default: 3)
After this many failures, message goes to topic.DLT automatically.
Server port: [PORT] ← change this (default: 8081)
Frontend origin for CORS: [URL] ← change this (example: http://localhost:3000)
Destination URL: [URL or empty] ← change this
If set, consumer will POST the result to this URL after processing.
Leave empty if the frontend polls directly from this consumer.

Evento model — must match exactly what the producer publishes

These two fields are always present — keep them:
- String transacao_id
- String timestamp

Add these business fields (must match the producer Evento fields exactly):
[LIST THE SAME FIELDS AS THE PRODUCER] ← change this
Example:
- String conta_id
- String cpf
- BigDecimal valor

- String tipo

Use @JsonIgnoreProperties(ignoreUnknown = true) on the class
so unknown fields do not break deserialization.

ResultadoEvento model — returned to frontend via GET /api/resultado/{id}

These fields define what the frontend receives after processing:
[LIST YOUR RESULT FIELDS] ← change this
Example:
- String id (store key — frontend uses this for polling)
- String status (example values: APROVADO, REJEITADO, PROCESSANDO)
- String mensagem (human-readable message for the user)
- String processado_em

Store key — used for frontend polling

The ResultadoStore saves results indexed by this key.
The frontend calls GET /api/resultado/{this_key} to retrieve the result.
Store key field: [FIELD NAME] ← change this
Example: conta_id
After processing, do: resultadoStore.salvar(evento.getConta_id(), resultado)
Frontend polls: GET /api/resultado/12345

Business logic — processarEvento() in EventoConsumerService

This is the only method where your project-specific logic goes.
Replace its body with:
[DESCRIBE WHAT THE CONSUMER DOES WITH EACH EVENT] ← change this
Example:
- Extract conta_id and tipo from evento
- If tipo is PIX: call pixService.processar(evento), set status APROVADO
- If tipo is TED: set status PROCESSANDO (async)
- If tipo is BOLETO: verify codigo_boleto, set status APROVADO or REJEITADO
- Build ResultadoEvento with the result
- Call resultadoStore.salvar(storeKey, resultado)

Error handling rules (do not change these):
- Throw IllegalArgumentException for invalid business data
  → goes directly to DLT, no retry
- Throw any RuntimeException for transient errors (DB down, API timeout)
  → retried N times (DLT retries setting), then goes to DLT

Important constraints — never violate these

1. Package declaration must be the first line:
   package com.example.consumer;

2. Do NOT use any Lombok annotation:
   No @Data, @Slf4j, @Builder, @RequiredArgsConstructor, @AllArgsConstructor

Use instead:
- Manual getters and setters
- private static final Logger log = LoggerFactory.getLogger(ClassName.class);

3. Do NOT include spring-boot-starter-security in pom.xml.

4. The main class must NOT be public:
class EventoConsumerApp (not public class)

5. Keep all classes in the single file EventoConsumer.java.

6. CorsConfig must always be included — without it the browser
blocks GET /api/resultado with CORS error.

7. Consumer must use manual offset commit:
AckMode.MANUAL_IMMEDIATE
enable-auto-commit = false
acknowledgment.acknowledge() called only after successful processing.

8. The DLT publisher uses the same broker config as the consumer.
The DeadLetterPublishingRecoverer routes to topic.DLT on the same partition.

9. The pom.xml artifactId must be: kafka-consumer
The pom.xml version must be: 1.0.0
The Dockerfile must copy: target/kafka-consumer-1.0.0.jar

10. The Dockerfile must use two-stage build:
Stage 1: maven:3.9-eclipse-temurin-17 to compile
Stage 2: eclipse-temurin:17-jre-alpine to run

Output format

Generate these 3 files completely, in this order:

FILE 1: src/main/java/com/example/consumer/EventoConsumer.java
Complete file, all 9 classes, ready to copy and paste.
Comments in Portuguese.

FILE 2: src/main/resources/application.properties
Complete file with "ALTERE APENAS AQUI" section at the top.
Comments in Portuguese explaining each property.

FILE 3: pom.xml
Complete file with all required dependencies.
No spring-boot-starter-security.
Comments in Portuguese.

Do not generate the Dockerfile — it never changes between projects.